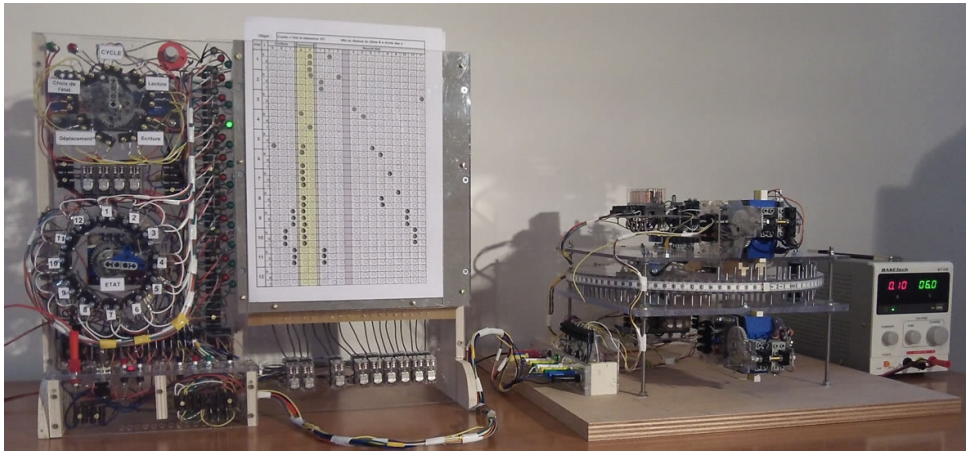


TD7

Décidabilité

Machine de Turing



Turing

M1101 M. Dalmau IUT de Bayonne

Décidabilité

Soit A le langage {a , ab , bba} et B le langage {baa , aa , bb} voici la règle du jeu :

- On forme un premier mot X en concaténant N mots de A, on peut prendre N aussi grand qu'on veut et mettre autant de fois qu'on veut les mêmes mots de A dans l'ordre qu'on veut.

  Par exemple M = aababababba constitué de a+ab+ab+a+bba est un mot correct pour N=5. Pour fabriquer X on a pris dans l'ordre les mots de rang 1,2,2,1,3 de A.

- Puis on forme un second mot Y en concaténant le même nombre N de mots de B pris dans le même ordre que ceux de A ayant servi à constituer X donc ici Y = baa+aa+aa+baa+bb (on a pris dans l'ordre les mots de rang 1,2,2,1,3 de B).

1. En suivant cette règle du jeu, peut-on trouver un mot X et un mot Y tels que X=Y
2. Même question avec les langages A={bbab, abba, b} et B={b, bb, bba}.

Commentaires

Ce problème est indécidable c'est à dire que si on prend deux langages A et B quelconques il n'y a aucune façon de savoir si on peut ou pas trouver X et Y sauf à essayer jusqu'à trouver une solution comme dans le 1er exercice.

Mais s'il n'y a pas de solution on ne peut pas le savoir, c'est peut-être seulement qu'on ne l'a pas encore trouvée comme c'est le cas du 2ème exercice si on abandonne à N=100 ou 200 ou 251.

Il faut donc essayer jusqu'à trouver X et Y mais comme N peut être aussi grand qu'on veut on ne peut jamais savoir !

On ne peut pas écrire un programme auquel on donne deux langages et qui affiche OUI ou NON selon qu'il y a ou pas une solution parce que l'algorithme de ce programme serait une boucle dont on ne sortirait que quand on a une solution (donc en affichant OUI) et on ne pourrait mettre nulle part le cas où on sort en affichant NON.

Remarque: pour résoudre le problème de 2ème exercice avec un programme il faut effectuer 16 000 milliards d'instructions machine soit presqu'une heure sur un CPU à 4,6 GIPS.

C'est ce problème de décidabilité qui est à l'origine des machines de Turing.

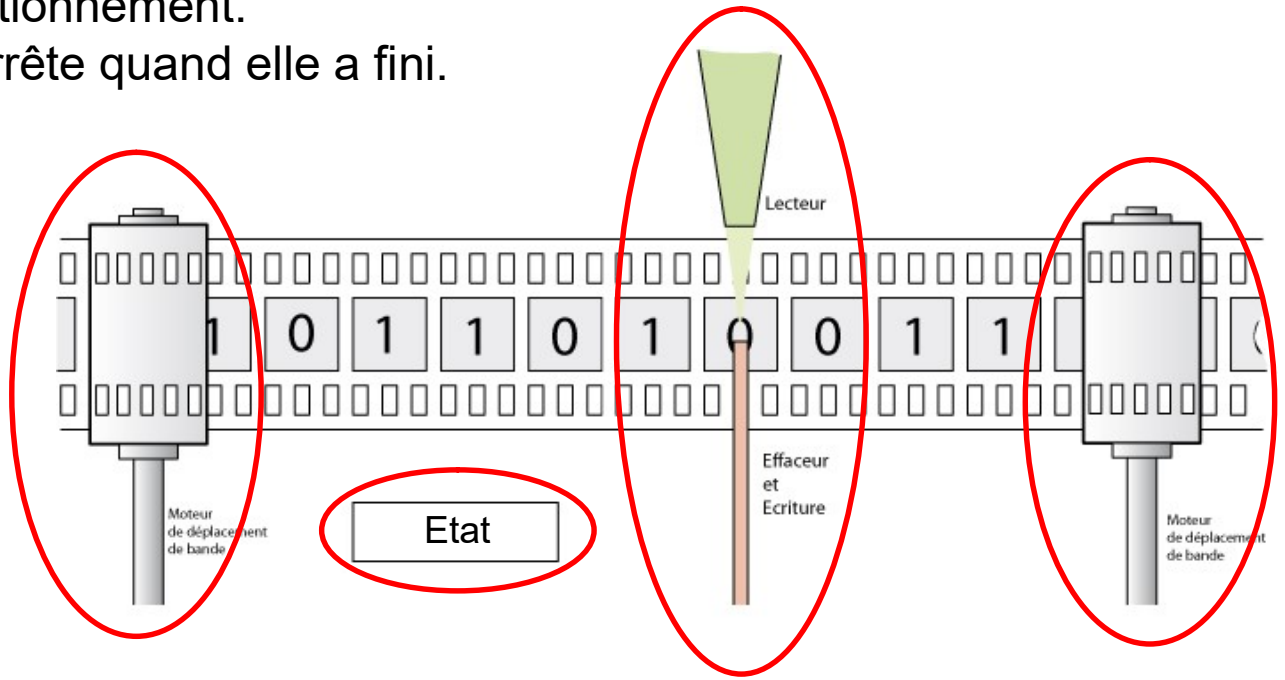
Machine de Turing

Principe : C'est une machine dotée d'une bande de papier infinie placée devant une tête pouvant lire et écrire des symboles sur cette bande. Quand elle écrit un symbole ça efface et remplace celui qui était écrit auparavant.

La machine peut faire défiler la bande d'un cran vers la droite ou vers la gauche (un cran correspond à un symbole).

La machine possède un état interne qui évolue au cours de son fonctionnement.

Elle s'arrête quand elle a fini.



Machine de Turing

Une machine de Turing est définie par :

- Un alphabet fini A = l'ensemble des symboles qu'elle peut lire/écrire sur la bande
- Cet alphabet contient un symbole spécial : l'espace (noté ε ou parfois .) qu'elle peut aussi lire/écrire sur la bande
- Un ensemble fini d'états Q
- Un état initial $q0 \in Q$

Son fonctionnement est le suivant :

1. La machine est dans un état $q \in Q$
2. Elle lit le symbole σ (élément de A) qui est devant la tête
3. Elle passe dans un état $q' \in Q$
4. Elle remplace le symbole σ par le symbole σ' (élément de A)
5. Elle déplace la bande d'un cran pour placer la tête devant le symbole de droite ou devant celui de gauche ou bien elle s'arrête
6. Si elle ne s'est pas arrêtée elle recommence en 1

Machine de Turing

Les symboles écrits sur la bande au départ représentent le problème à résoudre (par exemple ce seraient les contenus des langages A et B dans l'exercice précédent). On suppose toujours qu'au départ la tête est placée devant le premier symbole à lire (ça évite seulement de devoir faire défiler la bande tant qu'on trouve des espaces ce qui n'a aucun intérêt mais est facile à faire).

Le fonctionnement de la machine est décrit par une table qui donne pour chaque état q et chaque symbole σ lu :

- l'état suivant q'
- le symbole σ' à écrire
- le sens de déplacement de la bande indiquant si la tête va se placer devant le symbole de droite ou devant celui de gauche (D ou G) ou bien l'arrêt de la machine (ARRET).

On peut aussi le décrire par un graphe d'automate.

Exemple

La table est donc la suivante :

Etat \ symbole lu	0	1	ε
q_0	q_0 / 0 / D	q_1 / 1 / D	q_0 / 0 / ARRET
q_1	q_1 / 0 / D	q_0 / 1 / D	q_0 / 1 / ARRET

Principe :

- On a un état correspondant à un nombre pair de 1 et un autre état correspondant à un nombre impair de 1
- L'état correspondant à un nombre pair de 1 est aussi l'état initial q0 puisqu'au départ on a zéro 1.
- Dans chacun de ces 2 états un 0 nous laisse dans le même état (puisqu'on n'a pas changé le nombre de 1) et un 1 nous fait changer d'état (puisqu'on est passé d'un nombre de 1 pair à un nombre de 1 impair ou le contraire).
- Quand on arrive à la fin du mot binaire on remplace l'espace qui suit le dernier bit par un 0 ou 1 selon l'état dans lequel on est.

Question

Soit la machine de Turing définie par l'alphabet { 0, 1} et dont l'état initial est appelé q0.

Sa table est la suivante :

Etat \ symbole lu	**0**	**1**	**ε**
q_0	q_0 / 0 / D	q_0 / 1 / D	q_1 / **ε** / G
q_1	q_0 / 1 / ARRET	q_1 / 0 / G	q_0 / 1 / ARRET

1. Que va donner cette machine de Turing pour une entrée égale à 000 ?
2. Que va donner cette machine de Turing pour une entrée égale à 011 ?
3. Que va donner cette machine de Turing pour une entrée égale à 111 ?

Compléments

Turing a montré que pour tout problème on peut trouver une table pour sa machine avec laquelle elle le résout et s'arrête s'il est décidable, tandis que, si le problème n'est pas décidable, elle ne s'arrêtera jamais.

Comme elle ne met aucun temps pour effectuer ses opérations ou elle trouve et s'arrête de suite ou elle ne trouvera jamais et ne s'arrête jamais

En d'autres termes elle peut résoudre tous les problèmes qui ont une solution. Ainsi on peut trouver une table pour résoudre le problème de l'exercice 1. Sur la bande on mettrait les 2 langages A et B et la machine écrirait V ou F selon qu'on peut ou pas trouver X et Y. Mais, comme ce problème est indécidable, en fonction des contenus de A et B soit la machine s'arrêterait immédiatement en écrivant V soit elle continuerait indéfiniment à chercher.

Turing a également démontré que le problème de savoir si, pour une machine de Turing donnée (par sa table), il existait ou pas une valeur que l'on pouvait écrire sur la bande pour laquelle cette machine ne s'arrêterait jamais est lui même un problème non décidable.

Trouver la machine de Turing

Définir la machine de Turing qui remplace un entier relatif binaire en complément à 2 par l'entier relatif binaire en complément à 2 qui est son opposé.

On utilisera la méthode de lecture directe de droite à gauche.

Remarque : comme au démarrage la tête est, par définition, devant le premier bit (à gauche) de l'entier il faudra commencer par déplacer la bande jusqu'au dernier bit (à droite) avant de commencer le traitement proprement dit de calcul de l'opposé qui se fait de droite gauche comme le faisait la machine qu'on a vue pour l'incrémentation.

Compléments

On peut faire le parallèle entre la machine de Turing et un ordinateur (Von Neumann)

- *UT : Elle ne connaît que 5 opérations (lire un symbole, écrire un symbole, avancer la bande, reculer la bande et arrêter la machine).*
- *UC : Le compteur ordinal correspond à l'état actuel*
- *Mémoire : Sa mémoire c'est la bande mais elle est infinie en revanche on ne s'y déplace que vers le suivant ou le précédent alors qu'une mémoire d'ordinateur permet un accès n'importe où*
- *UE : Ses périphériques se résument à la bande qui sert d'entrée et de sortie d'information (on y rentre les valeurs à traiter et on y lit le résultat)*

Mais le programme est contenu dans la table de transition. Comme cette table fait partie de la définition de la machine de Turing ça veut dire qu'une machine de Turing est définie pour résoudre un problème précis et qu'elle ne sait faire que ça. Si on fait une machine de Turing pour ajouter un bit de parité à un mot binaire elle ne sert qu'à ajouter un bit de parité à un mot binaire.

Donc cette machine n'est pas universelle puisqu'elle ne sait faire qu'une seule chose. Pour faire une machine de Turing universelle il suffit de faire une machine de Turing capable de remplacer n'importe quelle machine de Turing.

***C'est facile, il suffit de faire une machine de Turing dont la table de transition est écrite sur la bande : en changeant cette table elle remplace n'importe quelle machine de Turing**. A chaque étape cette machine parcours la table qui est sur la bande pour savoir ce qu'elle doit faire (état suivant, symbole à écrire et déplacement de la bande ou arrêt).*

Compléments

On peut alors remarquer que la machine de Turing universelle peut :

- Modifier son propre fonctionnement en écrivant sur la bande là où se trouve la table;
- Suivre un fonctionnement (défini par une table) dont le but est d'écrire sur la bande une autre table qui définira le fonctionnement qu'elle aura ensuite.

De la même façon la mémoire d'un ordinateur de type Von Neumann contient les valeurs à traiter **ET** les programmes (c'est l'équivalent de la bande d'une machine de Turing). Il peut donc modifier ses programmes (en pratique ça s'utilise rarement, sauf pour protéger du code contre le désassemblage ou dans les virus auto-décryptés) et il peut aussi en écrire d'autres qu'il fera après (en pratique c'est ce que fait un compilateur).

Par ailleurs il est très facile d'écrire un programme d'ordinateur qui simule une machine de Turing (on en trouve sur le web) et même qui simule une machine de Turing universelle.

Un ordinateur est donc une machine universelle aux limites suivantes près :

- Ses opérations prennent du temps (exemple du craquage de clés trop long pour être utilisable cf. calculs de temps d'exécution du TD 3).
- Sa mémoire n'est pas infinie comme la bande de la machine de Turing. Avec les unités de stockage on peut se débrouiller (utiliser des fichiers plutôt que des variables) mais leur taille n'est quand même pas infinie et, en plus, le temps d'exécution du programme augmente considérablement $\Rightarrow$ on retombe sur la limite précédente.